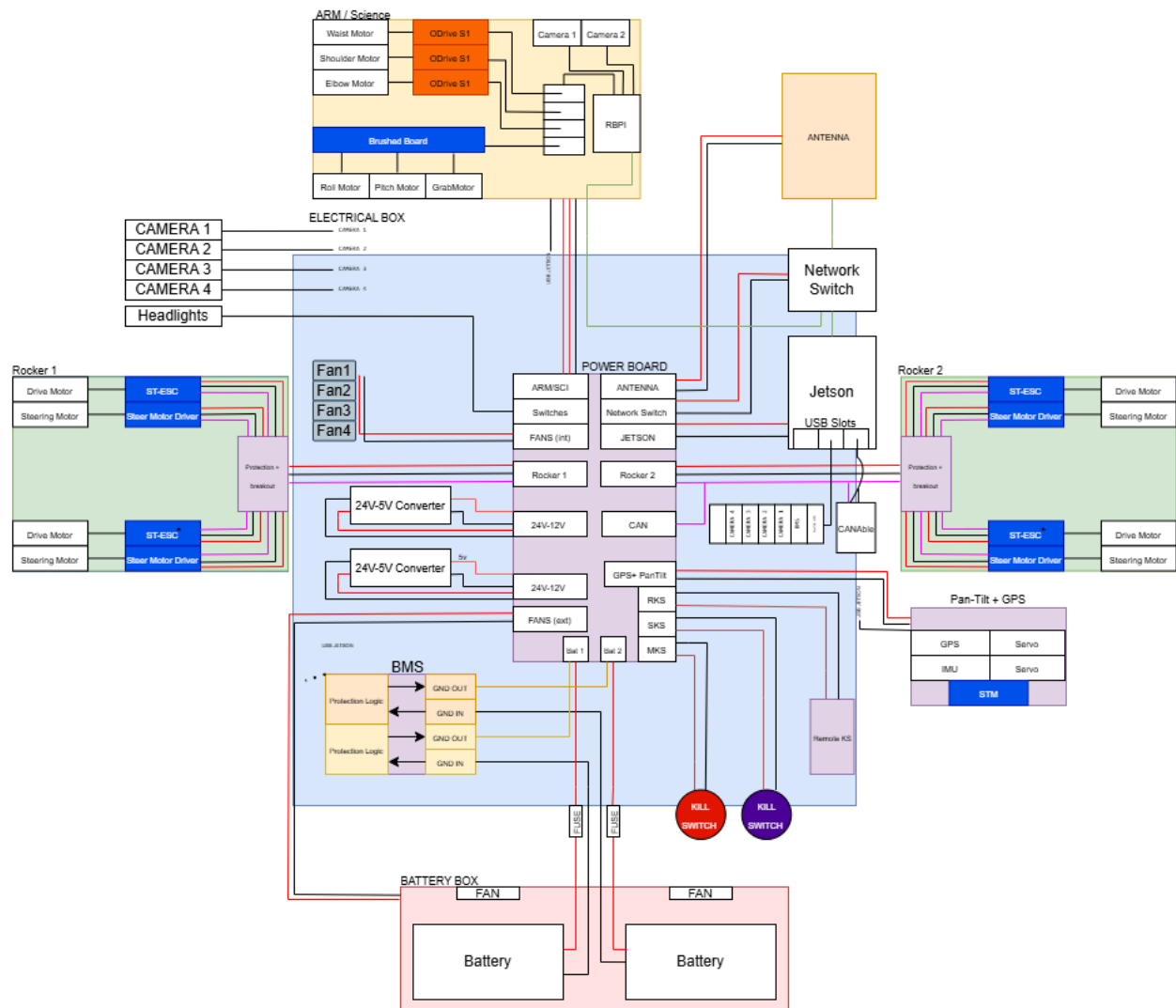


Rover 2025 Drive & Arm STM32 ESC Documentation

James Di Scullo

System High Level Overview



Drive Overview

One of the most significant changes from the 2024 Rover to the 2025 iteration is the redesign of the drive subsystem. The 2025 Rover now features a dual-rocker configuration, with each symmetrical rocker housing four motors, resulting in a total of eight motors in the drive system. Focusing on a single rocker (as the configuration is mirrored), each end is equipped with a brushless DC (BLDC) drive motor embedded within the wheel hub. These motors are responsible for providing propulsion via precise speed control. Additionally, each wheel is paired with a secondary DC motor mounted vertically, which functions as a steering motor to adjust the wheel's orientation.

Motivation behind B-G431B-ESC1 selection

In the previous year, our team utilized ODrive S1 motor controllers to manage the four BLDC motors in our drive system. While these controllers were effective in operation, we encountered two major limitations: limited customizability and a high cost for replacements and spare units. As a result, we began searching for an alternative BLDC motor driver that could be standardized across all rover subsystems, offering a balance of flexibility, cost-effectiveness, and seamless integration.

This led us to the B-G431B-ESC1, a motor control development board from STMicroelectronics, which stood out to us for several key reasons:

High Customizability

- The board allows us to develop and modify low-level firmware, enabling the implementation of custom control algorithms tailored to our rover's needs.
- It supports custom connector layouts, allowing us to standardize connections across all subsystems for streamlined integration and maintenance.

Robust Software Ecosystem

- The B-G431B-ESC1 is fully supported by a suite of STMicroelectronics tools, including MC Motor Pilot, Motor Control Workbench, STM32CubeIDE, STM32CubeMX, and STM32Programmer.
- Comprehensive open-source documentation facilitates debugging and simplifies development, especially when dealing with hardware-level issues.

Versatility and Performance

- The board supports multiple communication protocols, giving us flexibility in how we integrate it with other rover systems.
- Its specifications accommodate high-current applications, making it well-suited for our rover's voltage and power requirements.

Additional Features

- The board contains a multitude of additional protection features such as temperature feedback, overvoltage, and undervoltage protection.
- Very small form-factor, making it suitable for being mounted in a variety of different places on our robot

Modifications and troubleshooting with the B-G431B-ESC1

While evaluating the B-G431B-ESC1 board and testing its capabilities, we were able to confirm many of the advantages previously outlined. However, through extensive hands-on integration and testing, we also identified several hardware limitations inherent to the board. These shortcomings, while not critical, prompted us to implement a series of hardware-level adaptations to better tailor the board to the specific requirements of our river systems.

The first modification we made was not a result of an issue, rather it was to integrate it with the rest of the electrical harness of the rover. This included adding XT-90 connectors to the power terminals of the ESC, an MT-60 connector to the phases which connect to the BLDC motors, and a small 3 pin connector for CAN communication as shown in Figure 3.

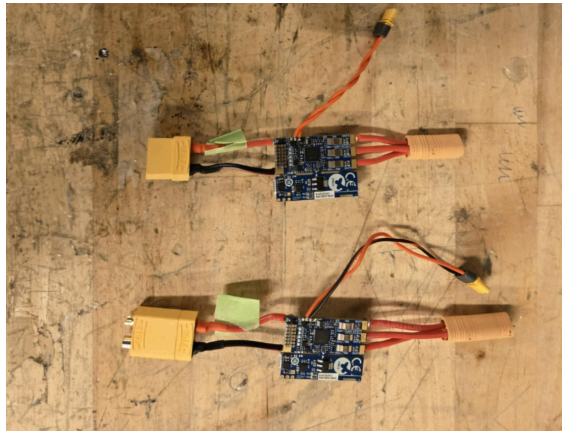


Figure 3. Initial modifications to the B-G431B-ESC1 board for interfacing with rover systems

Following these modifications, through using the board extensively in testing, we realized a recurring issue, the CAN pads on the board itself would frequently detach (CANH and CANL pads), preventing being able to communicate with the board via CAN. As a result, to solidify the connection between the wires and the CAN pads of the

board, a connector was soldered onto the pads, in an attempt to secure a stronger connection and reduce the likelihood that the pads would fall off the board.

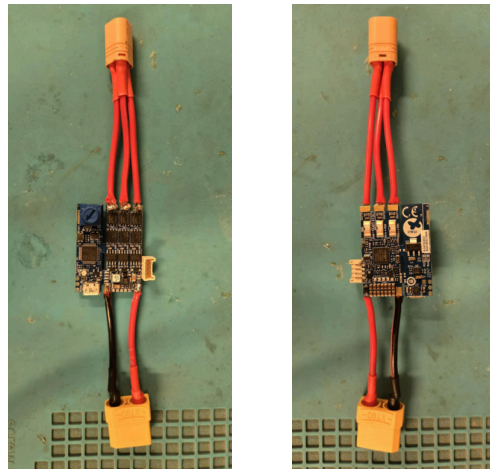


Figure 4. Addition of a connector on the CAN terminals in an effort to solidify connection

To further support the connector, a case was also designed and 3D printed to not only support the CAN wires themselves, so that they would not pull on the CAN connector, but also support the newly added connector to relieve strain on the pads of the PCB.

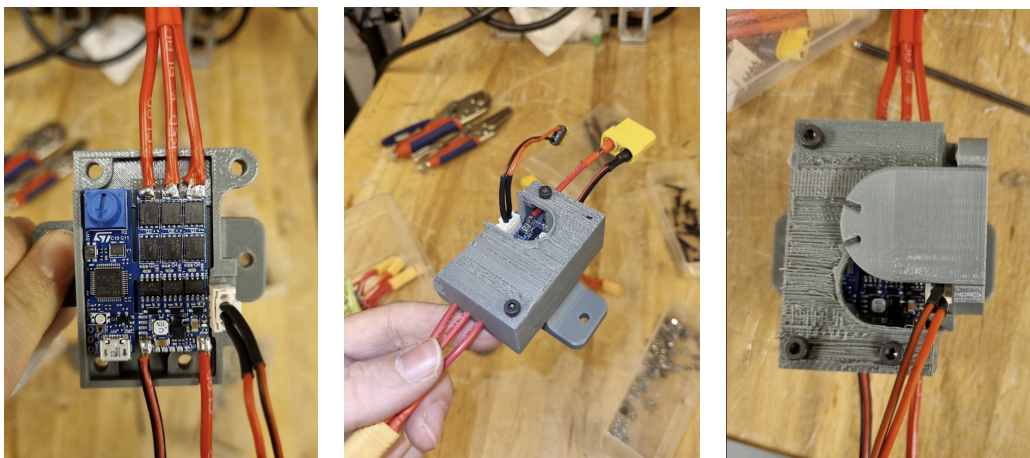


Figure 5. 3D Printed Case designed to help protect and support the ESC board, focusing primarily on the CAN connector.

However, despite these attempts to reduce the likelihood of the CAN pads falling off, they continued to do so after extensive usage, and frequent changes to the CAN harness of the rover. Thus, one final modification was made to the B-G431B-ESC1 boards in order to ensure that the pads remain secure, despite the frequent handling expected through experimentation and debugging of the rover drive system. The solution that was chosen was to pot the ESC in a resin material that would cure overnight, and encapsulate the board providing strong mechanical strength. Furthermore, knowing that the resin could lead to thermal issues, two heat sinks were added to the power components of the ESC, in order to dissipate heat. With the connector fully secured by the resin, and the heatsinks to dissipate the heat produced when running, the current configuration shown in Figure 6 is what we are currently using to test the B-G431B-ESC1 boards.

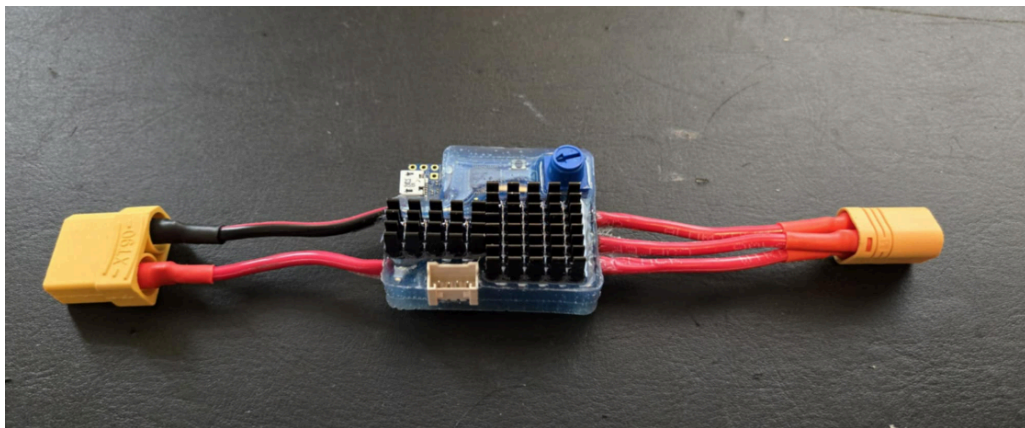


Figure 6. Finalized implementation for the B-G431B-ESC1 boards on the rover, featuring blue resin, and the heat sink on the power MOSFETs of the board.

Note that this is what an esc looks like for the drive system. The ones for arm follow a similar process, but use different connectors

Arm / Drive ESC Walkthrough

This section encapsulates the entirety of the drive/ arm firmware, including what are the more important files, how to run each script, and what they are doing. This section first starts off with a

walkthrough on how to generate all the files that are required in order to use the B-G431B-ESC1 esc to control a motor. Following this, it involves a breakdown of all the firmware files, noting the important sections in each relevant file, and where are some points that can be adjusted or improved upon for the coming years.

ESC Walkthrough: Setting up the ESC

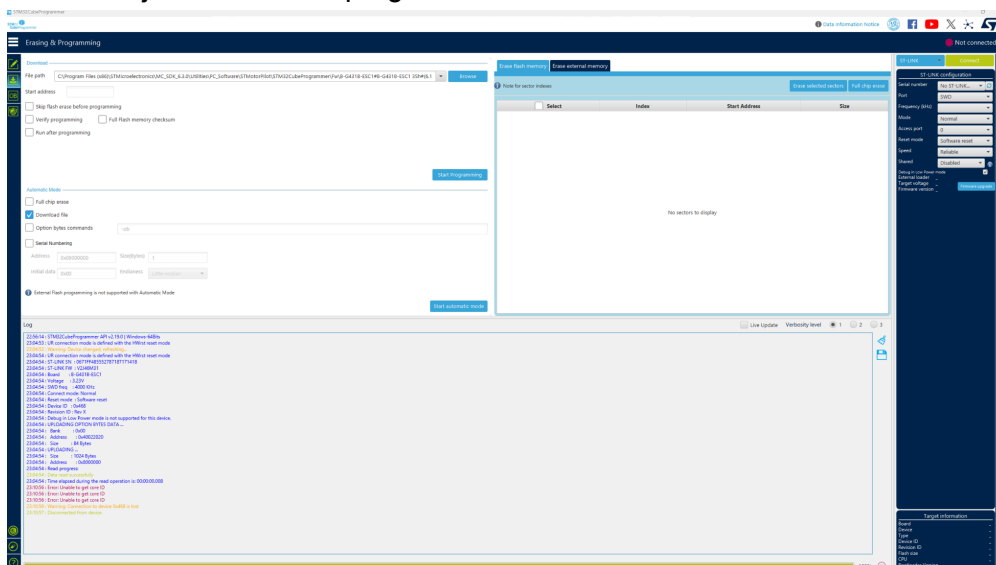
STEP 1: Profiling the motor

The first step towards being able to control a motor with an esc is to profile the motor. This involves getting different motor parameters which enables the esc to best control the current motor it is using, and to properly monitor the currents and voltages to protect the motor should anything weird happen. On the github, if you pull the existing code you don't have to worry about this step since I already profiled the motors, but its good to know the process in case we buy new ones 🤪(📊🔌🔧).

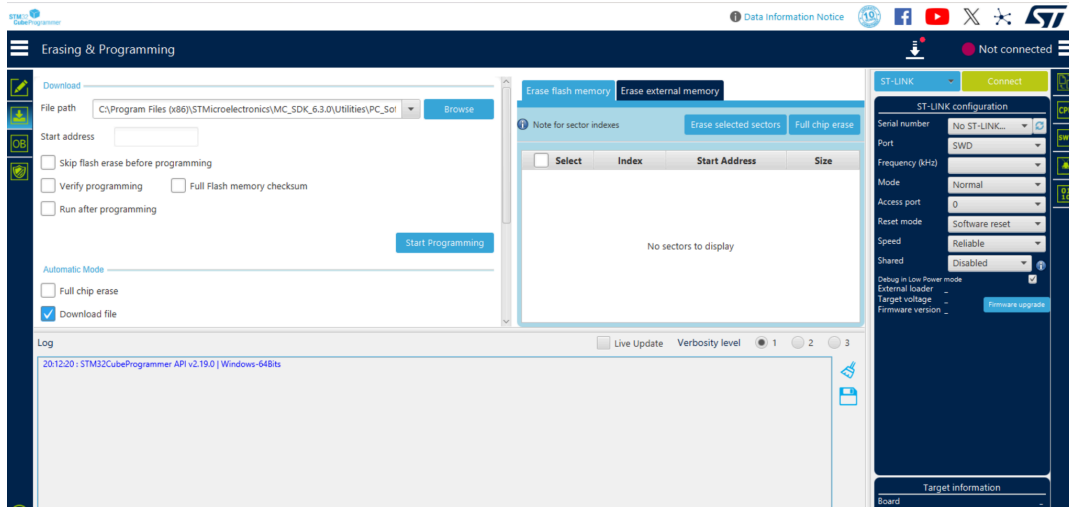
1. The first thing you need to do is flash the esc with the code needed to run the motor Pilot Application, which will profile the motor for you. You will get the firmware that you need to flash the ESC with in the following Link:

<https://www.st.com/en/development-tools/stm32cubeprog.html>

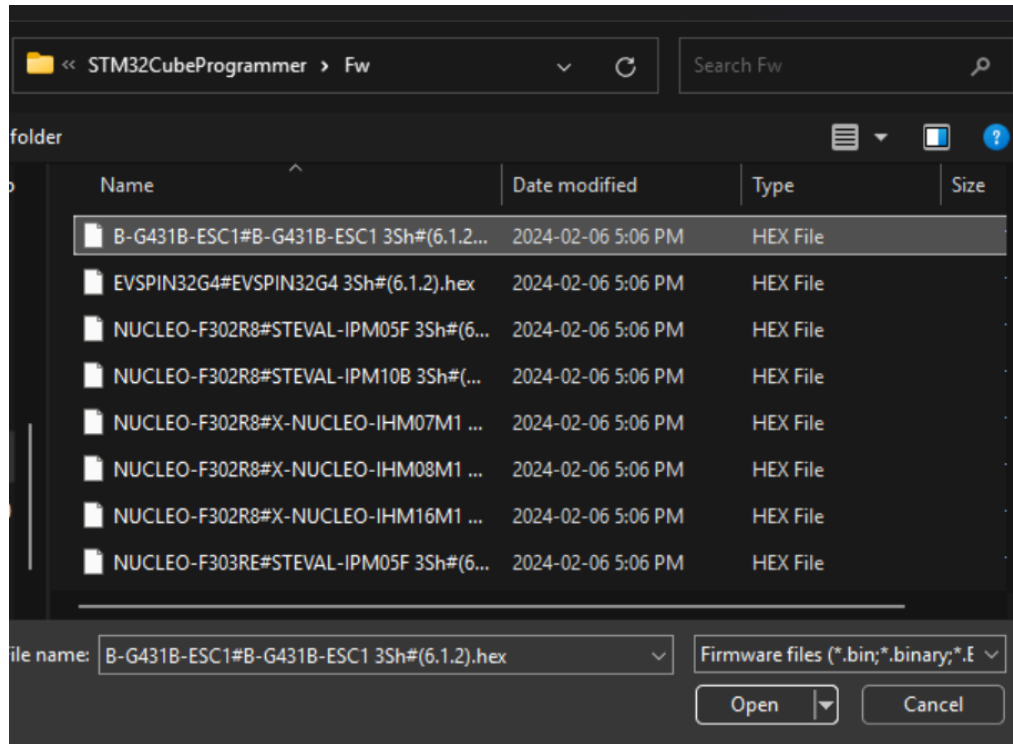
2. Open the cubeprogrammer and you should see something like the screen you see below. Once you are here, click connect with the ESC plugged in and it should connect. Otherwise just refresh the program.



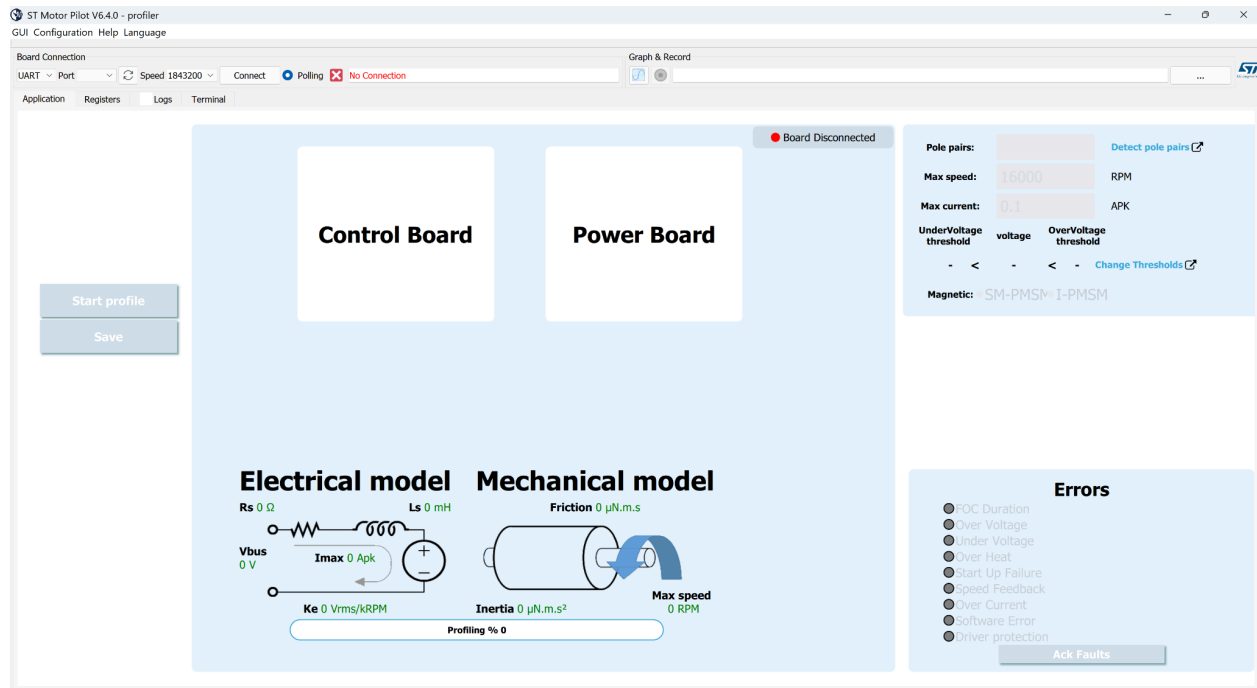
3. From this point click on the second option on the lefthand menu, and you will see this screen



4. Now click on open file, and then click this file at the following path in the image



5. Now click “start programming” and then wait for the firmware to upload.
6. Close the platform and unplug the esc (otherwise there are sometimes connection issues) and now open STMotorPilot. Once it opens click launch profiler



7. Once you are on this screen, just insert all of the motor specs on the right panel from the datasheet (if you don't know pole pairs follow the instructions there)
8. Plug back in the ESC and click on connect (may need to check COM port if not automatic)
9. Click Start profile! Be careful, it will spin the motor very fast near its highest RPM so make sure the shaft is free to spin a lot, and that the motor is being safely secured.
10. Download the specs somewhere and then your done:)

STEP 2: Tuning Parameters in the Motor Workbench

The Motor control workbench essentially gives you a GUI to easily modify the parameters of the ESC and motor, that way you don't need to scavenge the code too hard later for these numbers and constants. Here are the following steps that I usually go through when I am trying to configure a new ESC project / check a current one and how it works:

On the left hand side of the screen, you should see a section that says "Stage", click here and then go through all of them changing these parameters as described below:

Motor:

Pretty much make sure that all of the motor specs match with the motor profile that you made in the previous section. If using an encoder (**we always do so just do this**) enable the Quadrature encoder, and **you need to check the PPR of the encoder!** Do **not** click the index pin.

Power Supply:

Make sure that the max application current corresponds to that of the motor, and change the bus voltage to 25.5V.

PWM Generation:

Leave this at default

Speed Sensing Mode Selection:

Put this on Quadrature Encoder

Current Sensing, Bus voltage monitoring, Temperature Sensing, Speed Sensing:

Leave all these on their defaults

Drive Settings:

This setting depends on whether the ESC is being used for arm or drive

For drive:

Make sure that Speed control is being used.

For arm:

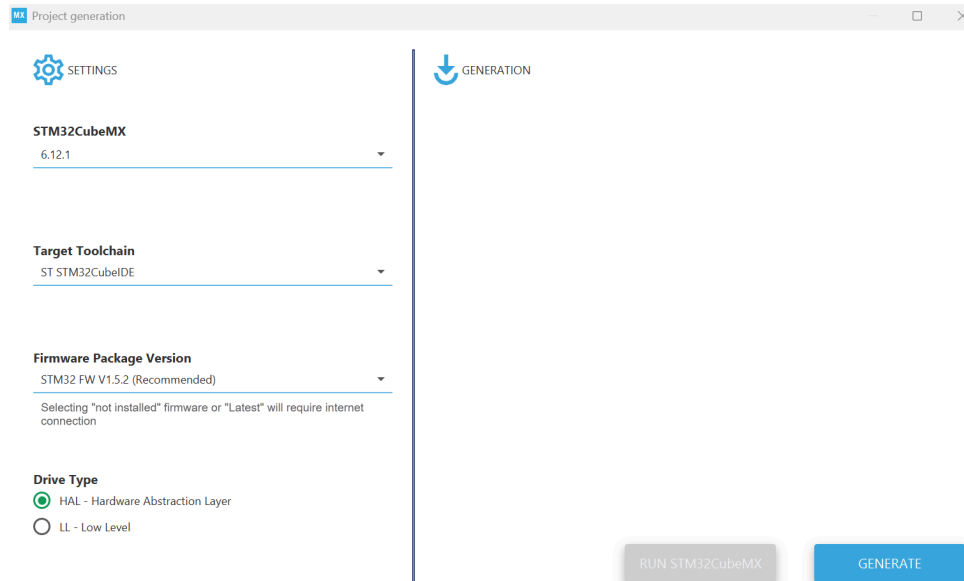
Make sure that position control is being used.

The rest can all be safely ignored, just click on OK at the bottom right and your all set here

Step 3: Generating the code

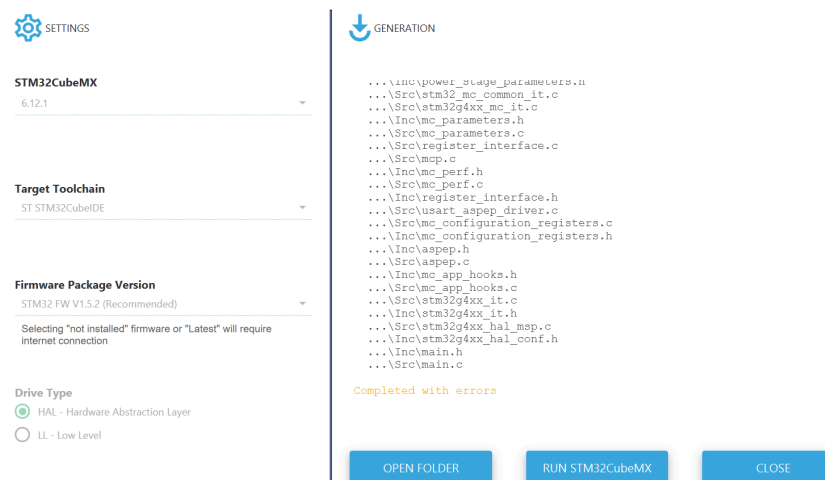
Once all the parameters from the Motor Control workbench have been set, it's time to allow the software to generate the code for your new motor project. The code that is automatically generated deals with incorporating predefined motor control schemes with the parameters that we specified in Step 2. This code does not dictate how the esc operates, it simply provides all of the libraries needed in order to control a motor to be under a specific project file.

1. Click on “Generate the project” at the top left side of the **MC Workbench**
2. Configure the settings such that they are as shown below, and then click on generate

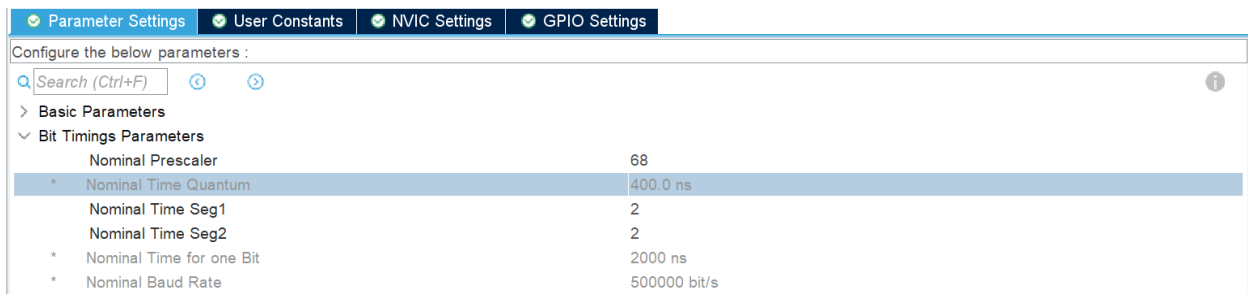


Note that on generation there might be “Attr not supported : Dmpu” errors which pop up. This is normal, and it's a known issue in ST's file generation. It is something that will be dealt with in the following step.

3. Once the generation is complete, you should see a screen like this appear, where you should then click on “RUN STM32CubeMX”. Cube MX will solve and deal with the previous issues which previously came from the code generation.

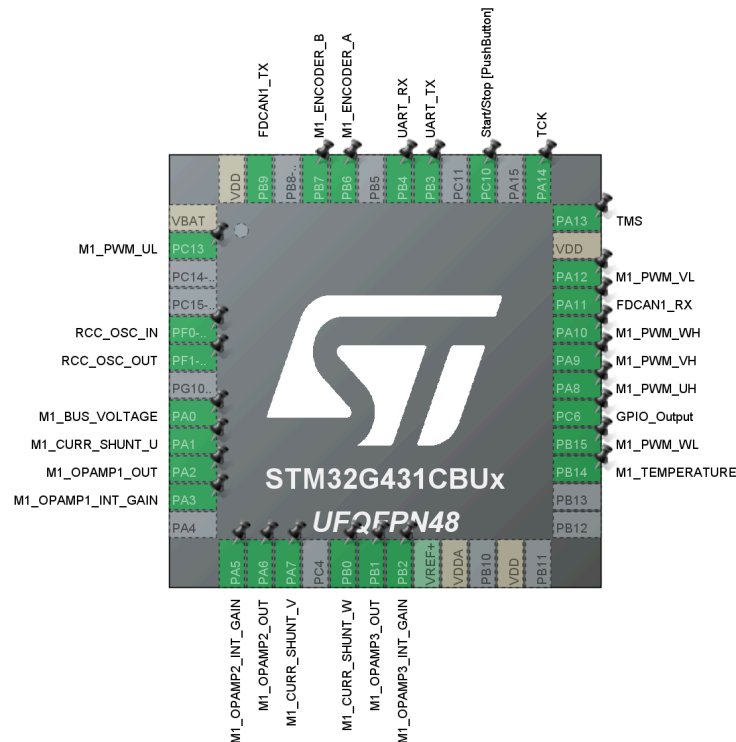


4. Now you should see the project ioc file open, and there should already be pins which are predefined for you, which were done in the Control Workbench. It is now time to enable the extra pins which we need in order to properly control the motor:
 - a. Click on PA11 and select FDCAN1_RX (this should automatically select FDCAN_TX in PB9)
 - b. Click on PC14, and select GPIO_Output (this will be used to select CAN terminations)
5. Now we will handle the clock configurations required for CAN. Note, this should already be the default, so just make sure all of these things are set as described below.
 - a. Select the "A->Z" Tab on the left side of the screen, and go to "RCC"
 - b. Once there, find "High Speed Clock (HSE)" and select "Crystal/Ceramic Resonator". What this essentially does is it forces the STM32 to use the external crystal on the board.
 - c. Click on "Clock Configuration" near the top of the page so that we can configure the clock frequency. In the "PLL Source MUX", select "HSE", which has the crystal frequency operating at 8Mhz. Now ensure that "PLLCLK" is selected in the "System Clock Mux".
 - d. Click on the "A->Z" tab on the left once more, click on "SYS" and in the "Debug" row, select "Serial Wire"
6. Now we will deal with the CAN baud rate configuration itself
 - a. Under the "A->Z" tab, click on "FDCAN", and enable it by clicking Activated (should already be ticked)
 - b. Now you should also see a "configuration" window below where the Activated icon is. Select "Parameter Settings", where we will now configure the baud rate. Here, in order to select a baud rate of 500k bits/second, go under "Timing Parameters", and change the nominal prescaler to 68. This is what the settings should look like when that happens:



7. Can interrupt setup
 - a. In the same window that you are currently in, navigate to the "NVIC Settings", and enable both interrupts there.
8. Now you need to configure the pin on the MCU that controls CAN termination, and for this purpose you need to select PC14, and then make this a GPIO output.
9. Almost there! Next thing is to make P6 an output, which will be used to control a status LED for debug.

10. Now once again the next steps change depending if the esc is being used for arm or drive. If its being used for drive, continue to the next steps (11 and 12). Otherwise skip to 13.
11. In the next step, we will configure the timing for UART, that way we can use print statements to debug. For this (always still in the ioc file), go to A->Z and then USART2 and change the baud rate to 115200 Bits/s
12. Generate the code! Now If you look back at the pinout, it should look something like this:

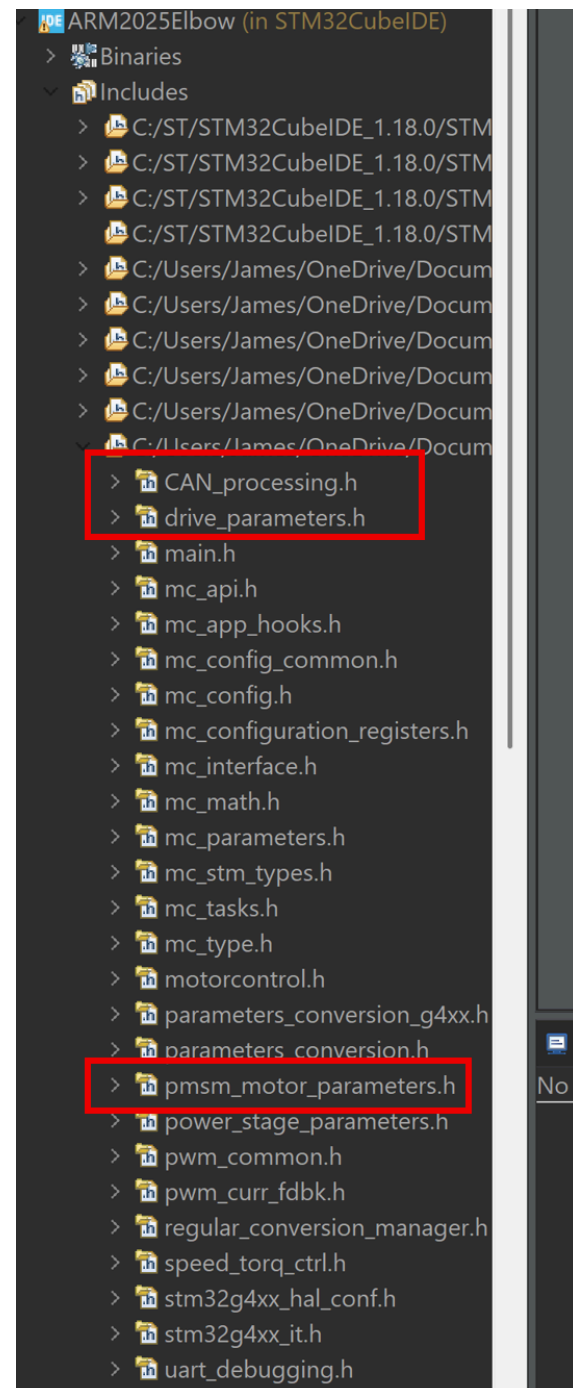
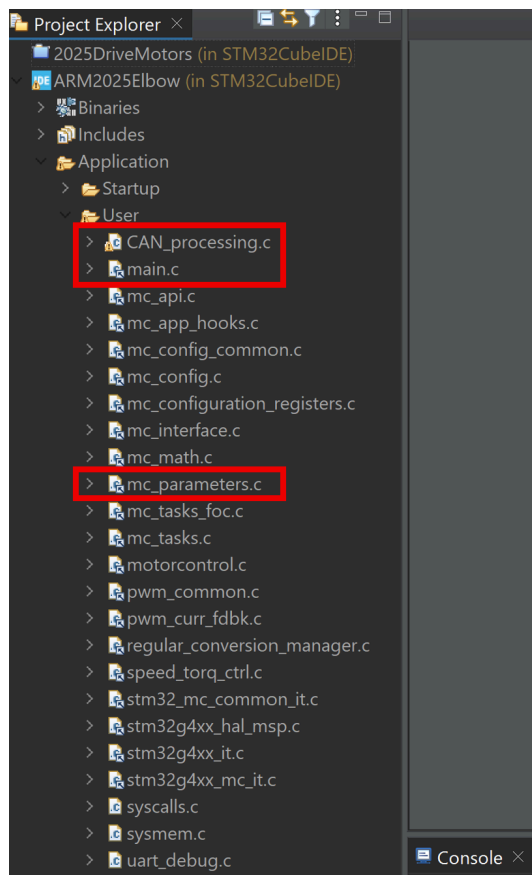


If this is the case, you have done everything correctly (yay) and the project is ready to be generated. In order to generate the project, click on “Generate Code” at the top right of the screen. If there is no option to generate code explicitly, then just save the file and it should prompt you to automatically generate the code anyways lol. While generating here is a short video which is explaining more details about how the generation process works, and how to avoid getting your code overwritten: <https://creations.mtdv.me/watch?v=STM32CUBEtutorials>

13. Not done yet... arm needs limit switches as well, so what we will do is repurpose some of the previously used uart pins which are exposed on the board for some limit switch detection. Select PB3 and assign it to “GPIO_EXTI3”.
14. Now on the left panel, in the A->Z part, please configure the settings as shown below. In the GPIO mode, select “External interrupt mode with Rising/Falling edge trigger detection”, and configure it with a pull-up.

ESC Walkthrough: General File Structure

There is a lot of generated code, as well as added custom files which are used together in order for the ECSs to properly control the motor. This section provides an overview of the important files to look out for / that will actually be useful when debugging or trying to understand a little more about what's going on. These files highlighted in red are the ones I was able to narrow down as the go to's in terms of what you really need to look at to understand what's going on, as well as which to modify when making the code better:)

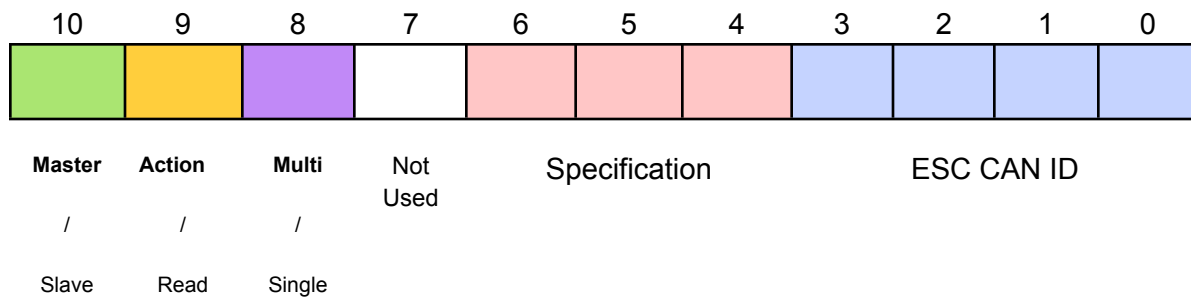


.c Files

CAN_processing.c

This file is a purely custom file which is used to parse through the CAN commands received on the CANbus and then process the command to control the motor. The start of the file is responsible for parsing through the can commands to decode the 11 CAN header bits to determine the purpose of the message that was sent. These 11 bits are encoded according to a custom instruction encoding, as shown below. It uses the same custom encoding as drive, which introduces some redundancy in the encoding when it comes to arm. In the CAN communication protocol, 0 is always dominant, thus the more important information is encoded closer to the MSB. Furthermore, lower number CAN headers have priority over higher numbers, therefore this was also kept in mind when creating the encoding scheme. For example a message sent by a master will always by default have priority over one from a slave device as 0 is dominant over 1, and thus the message will also have a lower number. Similar logic was selected for the ordering of the other pieces of information. In the breakdown below, **bold items** mean the bit is 0.

CAN Header breakdown:



	Master / Slave	This bit is used to indicate whether the message received was sent by a master (0) or a slave device (1). In the context of drive and arm, all of the ESCs are slave devices, thus this bit allows any ESC to immediately disregard any message from another one since an ESC never needs to communicate with another. In a similar way, it gives the master (in our case the CANable) priority over any message from another ESC if the bus messages occur at the same time.
	Action / Read	This bit indicates whether the esc must perform an action (0) or a reading to provide information (1) about its current state
	Multi / Single	This bit indicates whether the message payload (data) is intended for multiple ESCs (0) or a single one (1). This is essential for the ESCs to determine the significance of the data part of the payload.
	Specification	The specification has 3 bits reserved, for 8 total possibilities. These bits are used to specify what action/read is desired from the esc. These options are also different depending whether an action or read command was issued.

	ESC CAN ID	The CAN ID are 4 bits long, for 16 total possible devices to be running on the same bus at the same time. This is used to differentiate the ESCs, so that they each have a unique identifier.
--	------------	---

Once the messages are broken down, a struct is filled out with the information about the message, with a type of “**ParsedCANID**”. From this point, the file contain many switch cases to determine the control flow based on the decoded message’s intent. Please read the comments in the code for a more detailed description of each function and the general flow of events. The drive code is relatively straight forward, so only the 3 functions for the arm are explained in some more detail here (just this section)

The three main functions which cause the arm to move are:

runSingleMotorPosition()

This function is used to control the motor given a target position. This function expects the inserted position to be in degrees, and will trave to the given position relative to wherever the motor was calibrated (ie, wherever start motor was called last). It also performs some checks on the inputted target to ensure that it is possible to reach that target without violating any of the arm mechanical restrictions.

runSingleMotorPositionIncrement()

This function is used to make the motor move in increments, by the angle in degrees specified. This function will still perform checks to see whether the final setpoint of the arm is within an acceptable margin, and otherwise does not move at all.

runSingleMotorPositionFollower()

This function is used when software is responsible for detailing exactly how the motor should move. In this case, commands are sent at regular intervals, and the arm motor will simply follow the different setpoints and will not set an internal trajectory. This can be dangerous if not done properly as there is essentially 0 seconds allocated for the arm to execute a certain motion, thus if large jumps in setpoints are inserted the arm will move very sporadically

main.c

Aside from all of the initialising that main.c does, the most important parts of this file is where the esc ID is assigned at the top of the file, and how the interrupts are handled. When it comes to the arm, there are three different types of interrupts which can occur. The first is when a CAN message is received, and the function that handles that is:

HAL_FDCAN_RxFifo0Callback(FDCAN_HandleTypeDef *hfdcan, uint32_t RxFifo0ITs)

All that this function does is it assigns the value “true” to a flag variable to let the code know a CAN message has been received. This flag is polled in the while(true) loop of the main.c file.

Note: There are some commented out alternate versions to this function that plays around with how many commands are stored in a buffer, although we have always used the one that is currently used in the most recent pushed file when writing this.

The other two interrupts can be triggered when a limit switch is pressed, and unpressed, and they are handled in the following:

HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)

As I am writing this, the limit switch interrupts are not fully integrated in the while(true) loop of the main.c file, although it will follow a very similar implementation. The respective associated function in the CAN_processing.c file for upper and lower limit switch hit functions should be called then.

Aside from those interrupts, the other part in main.c which particular attention should be placed is the FDCAN initialization function:

MX_FDCAN1_Init(void)

More specifically, how the filters are set up must be checked. If ever there are issues communication over CAN after the IOC has been reset, this is the very first place to check.

mc_parameters.c

When it comes to this file, the only thing that is of concern are the OVP and OCP protection limits from the DAC:

.DAC_OCP_Threshold = 10000,

.DAC_OVP_Threshold = 25500

Ensure that these are properly set so that the ESC does not fault 😊

.h Files

The table below outlines the main header files which are consulted more often, as well as what pieces of information you can alter in each one of them. Please note that lots of the information

in these header files come from the configuration which was set up in the Motor control Workbench. This is just a much easier way to alter these numbers instead of going back and regenerating two layers worth of code which could be long. Just keep in mind that if ever you do change the MCDK and regenerate the project, you will have to be sure to remain consistent with the changes that were made.

CAN_processing.h	This header file contains all of the information needed for the decryption/ encryption of CAN commands, as well as function prototypes.
Drivce_parameters.h	This header file contains the PID tuning values, which can be modified much more easil than through the MCDK.
Pmsp_motor_parameters.h	This header file has the information about the encoder, more specifically the PPR

Future Improvements

There are many future improvements for these escs, notably in how quickly they are receiving/ sending data. I would like a transition from CAN to CAN FD to increase our bit rate from 500kbit/s to 4Mb/s. This would allow more telemetry to pass back to the rover, and an even smoother experience. Better PID tuning of the wheels would also be ideal, notably reducing the amount of KI which is present, although for the current set of wheels a higher KI is needed (this must be tested). All in all it works pretty well so yeah and if you have questions lmk.

Note that the arm firmware is changing a lot in 2026...new update on the way